

[← Go Back](#)

Hardware

UNO R4 WiFi

Tutorials

[Arduino UNO R4 WiFi ADC Resolution](#)[Arduino UNO R4 WiFi CAN Bus](#)[Arduino UNO R4 WiFi User Manual](#)[Arduino UNO R4 WiFi Arduino Cloud Setup Guide](#)[Arduino UNO R4 WiFi Digital-to-Analog Converter \(DAC\)](#)

Debugging the Arduino UNO R4 WiFi

[Arduino UNO R4 WiFi EEPROM](#)[UNO R4 WiFi Custom Firmware Upload to ESP32 \(Advanced\)](#)[Using the Arduino UNO R4 WiFi LED Matrix](#)[Arduino UNO R4 WiFi OPAMP](#)[Arduino UNO R4 WiFi Qwiic Connector](#)[Getting Started with UNO R4 WiFi](#)[Arduino UNO R4 WiFi Real-Time Clock](#)[UNO R4 WiFi Stack Trace Debug Runtime Errors](#)[UNO R4 Capacitive-Touch Tutorial](#)[Arduino UNO R4 WiFi USB HID](#)[Home](#) / [Hardware](#) / [UNO R4 WiFi](#) / [Debugging the Arduino UNO R4 WiFi](#)

Debugging the Arduino UNO R4 WiFi

Learn how to debug the UNO R4 WiFi with the Arduino IDE.

Author · Hannes Siebeneicher · Last revision · 10/07/2025

Debugging is the process of identifying and fixing errors in your code. It's a vital skill for anyone writing code especially when dealing with microcontrollers like those on your Arduino.

In this article, we will focus on using the Arduino IDE's built-in debugger, which can be used to debug the UNO R4 WiFi over USB. We will go through how to:

Enable debugging mode in the Arduino IDE

Create breakpoints for where we want to inspect the code

Monitor the value of a variable at a specific point in the program

Note that you **do not** need an external debugger to follow this tutorial.



To learn more about general debugging principles, you can read up on the topic [here](#).

Goals

The goals of this tutorial are:

learn about the basics of

ON THIS PAGE

Goals

[Hardware & Software Needed](#)[Debugging](#)[Connection](#)[Software](#)

+

[Set Correct Programmer](#)[Setting Breakpoints](#)[Start Debugging](#)[Help](#)

learn how to set up the Arduino IDE to debug an Arduino sketch.

Hardware & Software Needed

[Arduino IDE](#)

[Arduino UNO R4 WiFi](#)

[UNO R4 Board Package](#)

Debugging

Debugging your Arduino project allows you to dive deep into your code and troubleshoot as well as analyze the code execution. You can gain full access to the microcontroller's internal registers, memory, and variables. This is especially helpful when working on more complex projects where understanding the code execution flow is crucial. With the Arduino IDE you can step through the code line by line, allowing you to analyze why your code might break at a specific point.

Connection

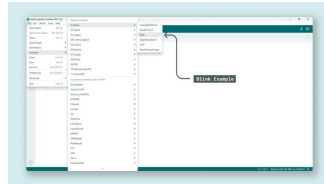
The only thing you need to do is to connect your UNO R4 WiFi to your computer using a USB-Cable.

Software

Setting up the Arduino IDE

First, if you haven't done it yet, install the [Arduino IDE](#). It's a good idea to

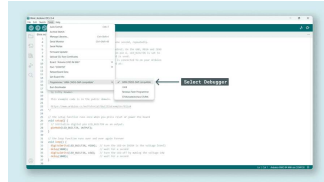
should by uploading the Blink example.



Blink Example

Set Correct Programmer

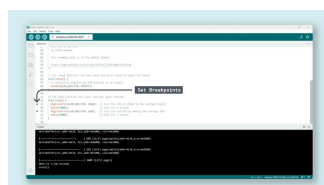
To access and debug the Arduino's MCU it's important that we set the correct programmer. Select **Tools > Programmer > ARM CMSIS-DAP compatible**.



Set Programmer

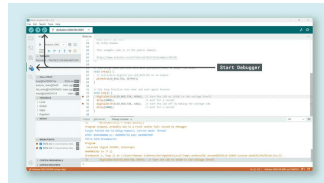
Setting Breakpoints

A breakpoint is an intentional stopping or pausing place at a specific point in the code. You can add them by clicking the sidebar next to your sketch, and you should see a red dot appear. You have now set a breakpoint.



Start Debugging

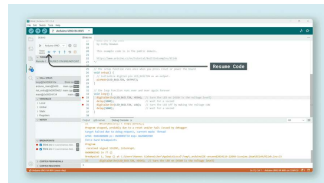
Now you are ready to start debugging.
Press **"Start Debugging"** next to the
"Upload button" or click the icon in
the left sidebar.



Start Debugging

You will see how your code is executed
and **stopped** at the line you set the
breakpoint. You can set as many
breakpoints as you want, depending
on where you want to stop your code.

To resume the code press the
"Continue" in the top left corner and
you will see how the code runs until
the it reaches the next breakpoint.



Resume Code

Suggest changes

The content
on
docs.arduino.cc is
facilitated
through a
public
[GitHub](#)
[repository](#). If
you see
anything

Need support?

[Help Center](#)
[Ask the Arduino](#)
[Forum](#)
[Discover](#)
[Arduino Discord](#)

License

The Arduino
documentation
is licensed under
the [Creative](#)
[Commons](#)
[Attribution-](#)
[Share Alike 4.0](#)
license.

Was this article helpful?

